Validation of Single-Precision (fp32) Arithmetic in magnum.np

Test and benchmark report of the performance changes (library code at commit `656a78b`)

Christoph Vogler*

2026-07-07

Abstract

This report documents the correctness and performance validation of the optional single-precision (fp32) mode introduced into magnum.np via `set_precision("single")`, together with the accompanying performance work (library code at commit `656a78b`; 24 commits on top of `main@ff6507a`). Validation comprises (i) exact-equivalence tests of all refactored kernels against the reference implementation, (ii) the full unit and demo test suites on CPU and on a Tesla V100 GPU, (iii) a dedicated fp32-vs-fp64 trajectory test (μMAG SP4-like), and (iv) a production-scale application benchmark: 200 ns field-free relaxation of a 240^3-cell soft-magnetic composite (13.8 M cells) at both precisions from bit-identical initial states. **Conclusion: for LLG time integration, fp32 reproduces fp64 observables within solver tolerance while halving both wall-clock time (1.98×) and memory (1.96×); double precision remains the default and remains recommended for tightly converged energy comparisons.**

Contents

1 Why fp32 is admissible: numerical background **2**

2 Test environments **2**

3 Exact-equivalence tests (refactoring safety) **2**

4 Test-suite results **3**

5 Dedicated fp32 correctness tests **4**
5.1 End-to-end trajectory test (`tests/unit/precision_test.py`) 4
5.2 Field-level agreement (demag benchmark checksums) 4

6 Application-scale benchmark: soft-magnetic composite **4**
6.1 Setup 4
6.2 Performance 5
6.3 Result agreement 5

7 Performance context of the branch (fp64 and fp32) **6**

8 Verification summary and scope **6**

9 Conclusions and recommendations **7**

A Data provenance **7**

*All numbers are measured, none are estimated unless explicitly marked.

1 Why fp32 is admissible: numerical background

The demagnetization field dominates the cost of finite-difference micromagnetics. Its evaluation splits into two numerically very different parts:

1. **Kernel setup (once per run).** The Newell formulæ evaluate 4th-order finite differences of the analytic cell-interaction functions f and g: 64 function evaluations with alternating signs per tensor entry. For distant cells the individual terms grow $\sim r^2$ while their sum decays $\sim r^{-3}$; at the switch-over distance ($p = 20$ cells) roughly 11 significant decimal digits cancel. This is fatal in fp32 (≈ 7 digits) and safe in fp64 (≈ 16 digits). *magnum.np therefore always evaluates demag/Oersted/vector-potential kernels in fp64*, independent of the selected run precision, and casts the final kernel to the run dtype afterwards (`demag.py`, `oersted.py`, `vector_potential.py`, `demag_nonequidistant.py`).
2. **Per-step evaluation (thousands of times).** Each field evaluation is FFT $\rightarrow$ pointwise multiply with the cached kernel $\rightarrow$ inverse FFT, plus local stencils (exchange, anisotropy, ...). None of these operations exhibit systematic cancellation; their relative rounding error stays at machine precision ($\varepsilon_{\text{fp32}} \approx 1.2 \cdot 10^{-7}$), i.e. two orders of magnitude below the default integrator tolerance (`RKF45`, atol $= 10^{-5}$) and far below model/discretization error. *This part carries the run precision.*

This mirrors the design of mumax^3, which runs fp32 throughout with high-precision kernel setup and reproduces fp64 codes on all μMAG standard problems. Additional fp32-specific safeguards in magnum.np:

- cell sizes are rescaled by $\min(\Delta x_i)$ during kernel setup to avoid fp32 overflow/NaN in intermediate powers;
- kernel cache files written at a different precision are cast on load, with a warning if precision would be lost;
- `set_precision()` refuses to run after the first `Mesh` is created, preventing silently mixed precisions;
- a hard-coded fp64 scratch allocation in `Minimizer_LBFGS` (which would have silently up-cast fp32 runs) was fixed.

Expected fp32 limits. fp32 is *not* recommended for (a) comparisons of nearly degenerate energies (barriers between similar states), (b) tightly converged minimizations with $\|\mathrm{d}m\|$ criteria near machine precision, and (c) any workflow relying on energy differences below $\sim 10^{-6}$ relative. The fp64 default is unchanged.

2 Test environments

3 Exact-equivalence tests (refactoring safety)

The performance work rewrote several numerical paths. Each rewrite was verified against the unmodified reference implementation *before* any fp32 testing, so that precision effects cannot be confounded with refactoring effects.

Trajectory-level equivalence (old vs. new code). Beyond kernel-level identity, the full time integration of the branch was compared against unmodified `main` on the V100: identical SP4-type problems (demag + exchange + external field, 520 accepted LLG steps) were integrated with both code versions at both precisions. The endpoint magnetization averages agree to machine-precision level, i.e. the optimizations do not alter the computed physics:

	CPU host	GPU host (vast.ai #44119668)
device	CPU only	Tesla V100-SXM2-16GB
PyTorch	2.12.1+cpu	2.12.0+cu126
Python	3.12	3.10 (vast.ai PyTorch image)
magnum.np	commit `656a78b`	identical tree (rsync)
baseline ref	`main@ff6507a`	identical

Table 1: Software/hardware environments. Note the V100 is a data-center GPU with fp64:fp32 throughput ratio 1:2; on consumer GPUs (ratio 1:32–1:64) the fp32 advantage is larger than reported here.

component	comparison	result	verdict
demag kernel (chunked setup)	vs. `main`, single chunk	bitwise identical	**pass**
	PBC $(2, 1, 0)$, 2-D meshes	bitwise identical	**pass**
	forced multi-chunk	rel. diff $\leq 1.7 \cdot 10^{-21}$	**pass**
non-equidistant demag (einsum/matmul)	vs. `main` triple loop	bitwise identical	**pass**
	vs. equidistant `DemagField`	rel. diff $2.6 \cdot 10^{-12}$	**pass**
`DemagFieldPBC` (cached factors, rfftn)	vs. previous fftn code	rel. diff $\leq 5.2 \cdot 10^{-16}$	**pass**
	odd sizes, degenerate dims	rel. diff $\leq 4.4 \cdot 10^{-16}$	**pass**
RuX exchange (cached coefficients)	vs. `main` per-call code	bitwise identical	**pass**
kernel evaluated on GPU vs. CPU	unit-test tolerances	ULP-level differences	**pass**

Table 2: Equivalence of refactored numerical paths against reference implementations (fp64, CPU unless noted).

4 Test-suite results

What a demo-suite "pass" asserts. The demo tests are quantitative regression tests, not smoke tests. Each test executes the *complete* simulation of its problem (energy minimization and/or full LLG time integration; 768 s total for all twelve on the V100) and then compares the produced trajectory *point-by-point at every logged time step* against stored reference data with atol = rtol = 10^{-3} (`torch.testing.assert_close`). For example, the SP4 test checks all three components of $\langle \vec{m} \rangle(t)$ over the entire switching trajectory against `ref/m_test.dat`; the FMR test compares the resonance response curve. The twelve problems are: μMAG SP4 and SP5, FMR, interface-DMI spectrum, domain-wall pinning, RKKY coupling, SOT, two Slonczewski spin-torque variants, soft-magnetic composite, self-induction (vector potential), and the minimizer. A V100 "pass" therefore means the optimized branch, running on GPU, reproduces the reference physics of all twelve problems — references generated with the original, unmodified code — within 10^{-3} at every time step. The demo tests contain no timing assertions; performance regressions are covered separately by the `bench/` suite (Sec. 7).

Known failures, root-caused.

- `rkky_test::test_intergrain_exchange`: `TypeError` due to a call-signature mismatch in `IntergrainExchangeField`; fails identically on unmodified `main`; unrelated to precision.
- `eigensolver_test::test_singlespin_*`: intermittent `ArpackError` ("no shifts could be applied"). The test solves a fully degenerate eigenproblem (all 20 requested eigenvalues identical); ARPACK convergence then depends on its random start vector, which is not seeded. Reproduced as flaky (8/8 passes on re-execution of the exact commits that had "failed"). Two by-products of this investigation were fixed on the branch: `Timer.__exit__` masked such exceptions with a secondary `TypeError`, and the precision test depended on the working directory left behind by `io_test`.

problem	precision	$\max_i \lvert\langle m_i\rangle_{\texttt{main}} - \langle m_i\rangle_{\text{branch}}\rvert$
SP4 200×50×1	fp64	$6.7 \cdot 10^{-12}$
SP4 500×125×2	fp64	$1.9 \cdot 10^{-12}$
SP4 200×50×1	fp32	$1.5 \cdot 10^{-8}$
SP4 500×125×2	fp32	$1.5 \cdot 10^{-8}$

Table 3: Endpoint agreement of $\langle \vec{m} \rangle$ after 520 LLG steps between baseline (`main@ff6507a`) and optimized branch, V100, identical problems. The residuals sit at the accumulation floor of the respective precision.

suite	platform	passed	failed	notes
`tests/unit` (baseline `main`)	CPU	130	1	pre-existing rkky failure
`tests/unit` (branch, final)	CPU	139†	1	same rkky failure only
`demos` integration (12 tests, incl. SP4/SP5)	CPU	12	0	
`tests/unit` (branch)	V100	137	2	rkky + flaky ARPACK
`demos` integration (12 tests)	V100	12	0	768 s wall-clock

Table 4: Test-suite outcomes. †includes 7 new tests added by the branch (copy-on-write materials, cache invalidation, fp32 end-to-end). The failures are analysed below and are unrelated to fp32.

5 Dedicated fp32 correctness tests

5.1 End-to-end trajectory test (`tests/unit/precision_test.py`)

An SP4-like problem ($25 \times 10 \times 1$ cells, demag + exchange + external field, s-state initialization) is integrated for 5 LLG steps in two fresh subprocesses, one per precision. Assertions (all **pass**):

- dtype propagates end-to-end (`state.dtype`, all field terms);
- $|m| = 1$ preserved everywhere at fp32 (tolerance 10^{-5});
- component-wise agreement of $\langle m \rangle(t)$ with the fp64 run within $5 \cdot 10^{-3}$ absolute;
- `set_precision` raises after mesh creation; `force=True` overrides.

5.2 Field-level agreement (demag benchmark checksums)

Within one precision, the per-component-FFT and batched-FFT demag variants agree to $\sim 10^{-8}$ relative on random magnetization (e.g. fp32: 65007157940 vs. 65007158587 absolute-field-sum on $128^2 \times 8$), confirming that fp32 summation error remains at the round-off floor even across $4 \cdot 10^5$-term reductions. (fp32/fp64 checksums are not directly comparable because the seeded random magnetizations differ per dtype.)

6 Application-scale benchmark: soft-magnetic composite

6.1 Setup

- Geometry: FCC-based random packing (`fcc_placer`, `example_4um_1um_70pd`); 240^3 cells at 27 nm (6.48 µm box, 13.8 M cells), 3-D periodic. Achieved packing $\varphi = 0.670$ (target 0.70), large:small = 73:27 (target 70:30), $d_L = 4$ µm, $d_S = 1$ µm.
- Materials: powder $J_s = 1.5$ T ($M_s = 1.194 \cdot 10^6$ A/m), $A_{\text{ex}} = 10^{-11}$ J/m, $K_u = 2$ kJ/m^3 with random per-cell easy axes, $\alpha = 0.1$; air $J_s = 10^{-8}$ T, $A_{\text{ex}} = 0$, $K_u = 0$.
- Physics: `DemagFieldPBC` (true periodic boundary conditions, cancelled surface charges) + exchange + uniaxial anisotropy; no external field; `RKF45`, atol $= 10^{-5}$, $\Delta t_{\max} = 0.1$ ns.

- Protocol: 200 ns relaxation from random per-cell magnetization. **Both precisions start from bit-identical states**: axes and m_0 are drawn once from a seeded fp64 NumPy RNG and cast to the run dtype.

6.2 Performance

	fp32	fp64	ratio fp64/fp32
wall-clock (200 ns)	4203.5 s (70.1 min)	8324.8 s (138.7 min)	**1.98**
CUDA peak memory	3.09 GB	6.06 GB	**1.96**
accepted RK steps	27 736	27 711	1.001
bytes per cell (measured)	224	≈450	2.0

Table 5: Composite relaxation, V100. The virtually identical step counts show the adaptive integrator is *not* forced to smaller steps at fp32, i.e. the local error estimate is not polluted by fp32 round-off.

The measured footprint implies maximum box sizes of $\approx 500^3$ (fp32) vs. $\approx 400^3$ (fp64) cells for this workload on a 32 GB card.

6.3 Result agreement

Field-free relaxation of a disordered system is chaotic: infinitesimal perturbations grow exponentially during switching events, so *point-wise* long-time agreement between any two arithmetic variants is impossible and is the wrong metric. The correct comparison is (a) trajectory agreement during the deterministic early phase, and (b) statistical equivalence of relaxed observables.

t	E_{fp32} [J]	E_{fp64} [J]	rel. difference
1 ns	$+1.813016 \cdot 10^{-12}$	$+1.813066 \cdot 10^{-12}$	$2.7 \cdot 10^{-5}$
2 ns	$+8.371534 \cdot 10^{-13}$	$+8.371537 \cdot 10^{-13}$	$3.4 \cdot 10^{-7}$
5 ns	$+3.144951 \cdot 10^{-13}$	$+3.146625 \cdot 10^{-13}$	$5.3 \cdot 10^{-4}$
10 ns	$+1.408864 \cdot 10^{-13}$	$+1.416453 \cdot 10^{-13}$	$5.4 \cdot 10^{-3}$
20 ns	$+4.907668 \cdot 10^{-14}$	$+4.991685 \cdot 10^{-14}$	$1.7 \cdot 10^{-2}$
200 ns (final)	$-2.244219 \cdot 10^{-14}$	$-2.289220 \cdot 10^{-14}$	$2.0 \cdot 10^{-2}$

Table 6: Total energy along the relaxation. Through $t \approx 5$ ns (thousands of RK steps) fp32 tracks fp64 to $10^{-7} \ldots 10^{-4}$; the growing late-time deviation reflects chaotic decorrelation into *different, statistically equivalent* multi-domain minima, not accumulating fp32 error (see text).

Evidence that the late-time difference is configurational, not numerical:

- both runs relax monotonically and keep drifting *in parallel* over the last 50 ns ($-2.11 \to -2.24$ vs. $-2.14 \to -2.29 \cdot 10^{-14}$ J);
- both final states are demagnetized to the same fluctuation floor, $|\langle m \rangle_{\mathrm{powder}}| = 3.4 \cdot 10^{-5}$ (fp32) and $1.5 \cdot 10^{-4}$ (fp64), i.e. the difference between the runs equals the natural magnitude of the observable itself;
- the divergence of $\langle m \rangle_{\mathrm{powder}}(t)$ between the runs saturates at that same floor ($\sim 10^{-4}$) from ≈5 ns on instead of growing;
- the integrator statistics (step counts, step-size evolution) are indistinguishable.

7 Performance context of the branch (fp64 and fp32)

For completeness, the V100 validation of the optimization branch itself (baseline `main@ff6507a` vs. branch, benchmark scripts in `bench/`):

benchmark	metric	`main`	branch	change
kernel init $256^2 \times 16$	GPU peak	992 MB	388 MB*	−61%
kernel init $512^2 \times 4$	GPU peak	1024 MB	445 MB*	−57%
kernel init $256^2 \times 16$	time	0.66 s	1.48 s*	+0.8 s (one-off)
MinimizerBB (fp64)	iterations/s	445	816	+83%
MinimizerBB (fp32)	iterations/s	467	797	+71%
SP4 500×125×2 (fp64)	steps/s	0.36	0.46	+28%
SP4 500×125×2 (fp32)	steps/s	0.61	0.87	+43%
demag $h()$ $128^2 \times 8$ (fp32)	ms/call	1.13	0.70	−38%

Table 7: Branch validation on the V100. *with `kernel_device` fix (kernels evaluated on the simulation device in bounded chunks); `kernel_device="cpu"` further lowers the GPU peak to the kernel size (214 MB resp. 252 MB) at the cost of slower setup. The fp32:fp64 SP4 ratio of 1.9 on the V100 (fp64 rate 1:2) grows substantially on consumer GPUs.

The batched single-FFT demag variant was evaluated and *rejected*: it is slower than the per-component FFT loop at both precisions on both CPU and V100.

8 Verification summary and scope

Table 8 consolidates all verification layers of this campaign.

verification layer	status
Refactored numerics vs. reference implementations (demag kernels, non-equidistant einsum, RuX coefficients, `DemagFieldPBC`)	bitwise / machine precision **pass**
Old-vs-new physics trajectories (SP4, V100, both precisions, Tab. 3)	$\leq 1.5 \cdot 10^{-8}$ after 520 steps **pass**
Unit suite, CPU (139 tests incl. 7 new)	1 failure, pre-existing on `main` **pass**
Unit suite, V100	2 failures: same pre-existing + root-caused ARPACK flak
Demos integration suite (12 end-to-end problems incl. SP4/SP5)	all pass, CPU and V100 **pass**
fp32 validity (dedicated trajectory test + 13.8 M-cell production run)	Secs. 5, 6 **pass**
Adversarial multi-agent code review (high effort)	10 findings → 8 fixed with regression tests, 2 documented
Performance non-regression (V100 + CPU benchmark sweeps)	1 regression found, fixed, re-confirmed **pass**

Table 8: Verification matrix of the performance changes (commit `656a78b`).

Explicitly outside the verified scope.

1. The project GitLab CI has not yet run on the branch (it repeats the unit and demo suites on the project runners) and should gate the merge as usual.

2. Two intentional behavior changes ship with the branch and must be communicated in the merge request: copy-on-write material parameters (chained or non-indexed in-place writes on never-written homogeneous parameters now require `Material.materialize()`), and the now-functional NaN guard in `LLGWithLESolver` (previously a no-op).

3. Deliberately unchanged, pre-existing issues: the `zero_origin` far-field kernel defect (pseudo-PBC images / far non-equidistant layer pairs) and the rkky unit-test failure; both are documented and proposed as separate fixes.

9 Conclusions and recommendations

1. **fp32 is validated for LLG time integration** (dynamics, relaxation, hysteresis-type protocols): trajectory-level agreement within solver tolerance during the deterministic phase, statistically equivalent relaxed states, unchanged integrator behavior — at 2.0× speed and 0.5× memory on a V100 (larger speed advantage on consumer GPUs).

2. **Kernel setup remains fp64 by construction** independent of the run precision; no user action required.

3. **fp64 remains the default** and is recommended for energy-barrier comparisons, tightly converged minimization, and eigenproblems.

4. Practical guidance: enable with `set_precision("single")` before creating the mesh; for long AC protocols (e.g. 3 µs at 1 MHz on 240^3 cells ≈ 21 h/V100 at fp32) start from a relaxed state and consider a loosened `atol` after a convergence spot-check.

5. Known, documented non-fp32 issues: pre-existing rkky test failure (`main`), unseeded-ARPACK test flakiness, and the pre-existing `zero_origin` far-field kernel defect for pseudo-PBC images/far layer pairs (behavior intentionally preserved bit-identically; fix proposed separately).

A Data provenance

- Code: library at commit `656a78b`, 24 commits on `main@ff6507a`; all equivalence/verification scripts run on 2026-07-07.
- Raw benchmark JSONs, run logs and composite results: `fcc_placer/example_4um_1um_70pd/results_v100/` (`validation/{base,opt,opt_fixed}`, `logs/`, `single/`, `double/` incl. `m_final.vti`).
- Benchmark scripts: `bench/` (this repository) and `fcc_placer/example_4um_1um_70pd/bench_composite_fp.py`.
- GPU: vast.ai instance 44119668 (Tesla V100-SXM2-16GB), total cost of the campaign < $1.